

Data Scaling

09 September 2026 02:47 PM

Database Scaling — Easy Notes

When an application grows, one database may become a **bottleneck**. We can use different techniques to handle more data and traffic.

1. Vertical Scaling

Increase the power of the same database server.

Before:

DB → 4 CPU, 8 GB RAM

After:

DB → 16 CPU, 64 GB RAM

Pros

- Simple
- Easy to implement

Cons

- Expensive at higher sizes
- Has a hardware limit
- Single point of failure

Remember:

Vertical scaling = **Bigger machine.**

2. Indexing

An **index** helps the database find rows faster without scanning the entire table.

Think about a book:

Without index → Search every page ✗

With index → Go directly to the page ✓

Example:

```
CREATE INDEX idx_user_email
```

```
ON users(email);
```

Good for **frequently queried columns**.

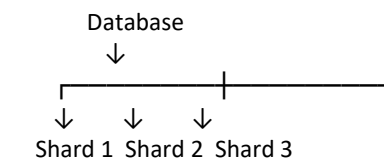
⚠ Too many indexes can slow down **writes**, because indexes also need to be updated.

Remember:

Indexing = **Find data faster.**

3. Sharding

Split one large database into **multiple smaller databases/servers**.



Example:

Users 1–1M → Shard 1

Users 1M–2M → Shard 2

Users 2M–3M → Shard 3

Why?

A single machine has limited:

- Storage
- CPU
- Memory

Sharding allows the system to handle much more data/users.

Remember:

Sharding = **Split data across machines.**

4. Vertical Partitioning

Instead of splitting **rows**, split **columns**.

Suppose we have:

Product

id
name
price
category
description
specifications
images
videos

We can split it:

core_product

→ id, name, price, category

product_details

→ description, specifications

product_media

→ images, videos

Now queries that only need name and price don't need to read the large media/details columns.

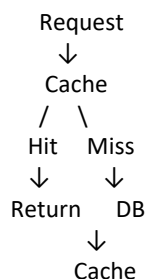
Remember

Sharding → split rows/data across servers

Vertical partitioning → split columns

5. Caching

Frequently requested data is stored in a **faster storage layer**.



Example:

A movie that millions of users request:

Movie details → Cache

Instead of hitting the database every time.

Benefits

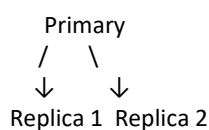
- Faster responses
- Less database load

Remember:

Caching = **Keep frequently used data somewhere fast.**

6. Replication

Create multiple copies of the database.



Usually:

Writes → Primary

Reads → Replicas

This helps with:

- Read performance
- High availability
- Disaster recovery
- Lower latency for users in different regions

Types

Synchronous

Write → Primary



Replica



Confirm



Success

More consistent, but slower.

Asynchronous

Write → Primary → Success



Replica

Faster, but replica may temporarily be behind (**replication lag**).

7. Materialized Views

Suppose a query is **very complex and frequently executed**.

Instead of calculating it every time:

Complex Query



Calculate every time **✗**

We calculate it once and **store the result**:

Complex Query



Pre-compute



Store Result



Fast Read ☒

That's a **Materialized View**.

Example

Daily sales:

Date | Product | Total Sales

Instead of repeatedly calculating:

SUM(quantity)

SUM(amount)

GROUP BY date, product

we store the result.

Remember

Materialized View = **Pre-computed query result**.

8. Data Denormalization

Normally, data is split across multiple tables.

Example:

users
posts
comments
likes

To get everything, we may need multiple **JOINS**.

Denormalization intentionally introduces **duplicate/redundant data** to reduce expensive joins.

Normalized:

users — posts — comments

Denormalized:

user_profiles

├── user data

├── posts

└── comments

This can make **reads faster**, but creates more duplicated data and can make consistency harder.